Selecting Reliable LLM Reasoning with Context-Conditioned Preferences

DASE7506 | Mini-project 2 | 27 October 2026

Official test accuracy: 83.94% (392/467). Validation: 93.61% (747/798); mixed-correctness validation groups: 87.83% (368/419).

1. Research question

Can the relative importance of reliability signals depend on the problem context? A candidate's integer answer, agreement with peers or concise reasoning can be useful on one kind of problem and misleading on another. Our original linear ranker used fixed coefficients for these signals. We test a small context-conditioned extension while retaining its efficient group-level objective and strict CPU/asset constraints.

Given four supplied candidates, the task is to select any candidate with a correct final numeric answer. The official parser and scorer determine correctness; reasoning steps need not all be correct. We fit only on the supplied training bank. Validation is for development and selection. Test references are used only for final scoring after freezing.

2. Data and baseline

The bank contains 4755 training, 798 validation and 467 test questions. Many training and validation groups have four correct candidates, making their overall accuracy uninformative about the difficult choices. We therefore report mixed-correctness validation accuracy on 419 groups alongside the official overall metric. Groups, not individual candidates, are the unit of cross-validation.

The original baseline concatenates 23 handcrafted features, their 23 group-normalized copies and the supplied 1024-dimensional trace embedding. The features cover text statistics, official-parser success, numeric answer properties, agreement among candidates, embedding geometry, candidate position and lexical overlap with the question. A standardized linear score is fitted with group-softmax set likelihood:

```
L_g = -log(sum_{j correct in g} softmax(s_g)_j).
```

All-correct groups contribute zero loss. This objective rewards selecting any correct candidate rather than treating all candidates as independent examples. The baseline achieved 367/419 on hard validation groups and the previously recorded 382/467 on test. Its original code and checkpoint remain preserved.

3. Context-conditioned method

For each question, average its four supplied embeddings to obtain a shared context vector. Subtract the training mean and project through a training-fitted whitened PCA basis. Keep eight context coordinates u. For each candidate, take its 46 handcrafted and group-normalized features h_j and form their outer product with u. Concatenate these 368 interactions with the original 1070 features:

```
s_j = w_base^T standardize(x_j) + w_context^T standardize(vec(u outer h_j)).
```

This is a bilinear context/candidate model. Equivalently, the coefficient on each handcrafted feature changes with the shared context. Unlike a generic hidden-layer network, this extension restricts nonlinearity to an interpretable interaction family. The shared context comes from trace embeddings; we do not claim it is a separate question encoder or that a PCA direction identifies a named mathematical topic.

Train with the original group-softmax objective, zero initialization and full-batch Adam: 1500 steps, learning rate 0.02, base L2 0.02. Stronger L2 of 1.0 on the interaction block keeps the correction small. All normalization/PCA statistics and coefficients are fitted on training only. The inference function loads these fixed assets and applies them directly to new bank inputs.

4. Development and ablations

Three-fold group CV, seed 7506, selected among eight contextual configurations: centroid/residual embedding interactions versus centroid/hand-feature interactions, dimensions 8/16 and extra-block L2 0.2/1.0. The baseline was included as a control. The 8-dimensional hand-feature interaction model won: 2096/2513 OOF hard groups versus 2084/2513 for the matched original-Adam baseline. Selection was written to disk before loading validation.

Method	Validation hard	Validation all
Majority answer vote, original experiment	73.75%	86.22%
Original linear group-softmax	87.59%	93.48%
Context-conditioned hand features	**87.83%**	**93.61%**
Shrinkage Gaussian quadratic residual fusion	86.87%	93.11%
Uniform correct-target softmax, CV-selected	86.87%	93.11%
Conditional RBF features, CV-selected	87.35%	93.36%
Delexicalized mathematical ngrams, CV-selected	86.87%	93.11%
Eight-member Bayesian-bootstrap contextual ensemble	87.11%	93.23%

Earlier development also tried GBM/MLP, PCA reduction, unrestricted TF-IDF, arithmetic consistency, number-origin coherence, pairwise GBM, answer clustering, candidate kNN and margin objectives. None improved on the original baseline; their detailed records remain in `code/EXPERIMENTS.md` and the original report. This continuation added residual geometry, question-conditioned preferences and executable-program transfer before the final optimization round.

The Gaussian alternative learns correct/incorrect covariances in training-fitted residual PCA space, giving a genuinely quadratic reliability score. It improved training OOF slightly but lost on validation. Alternative convex objectives improved OOF more strongly but also lost on validation, illustrating why an internal CV gain alone is insufficient. RBF and restricted text features failed to add reliable signal at their selected complexity. The bootstrap model averaged all eight predeclared members; no favorable seed was selected.

Program transfer used training reference reasoning rather than just final-answer labels. We extracted 3846 arithmetic programs by replacing question quantities with variables and composing reference `<<expression=result>>` steps. A number-normalized question TF-IDF index proposed programs for new numbers. Training OOF found no useful override threshold, so the selected threshold disabled the override. This was not used in the final model. Its failure indicates inadequate retrieval of transferable structure, not that all symbolic supervision is ineffective.

5. Frozen evaluation and interpretation

The contextual model rescues two hard validation errors and introduces one, a net gain of only one question (0.239 percentage points). This is insufficient evidence of a robust validation breakthrough. The unadjusted OOF paired comparison is also exploratory because those folds selected the configuration. We therefore avoid claiming statistical significance from development results.

After the owner authorized a final test run, we selected the best method using validation only, copied the checkpoint into `runs/final_candidate/`, recorded source/asset hashes and configuration in `runs/candidate_freeze.json`, and confirmed its official validation result. We then ran the official test prediction and independent scorer once. No subsequent method change or search used the result.

Frozen method	Official test
Original previously evaluated model	382/467, 81.80%
Context-conditioned model	**392/467, 83.94004282655246%**

The new model gains ten correct test selections and 2.14 percentage points. The larger test gain than validation gain demonstrates that tiny validation differences do not predict the exact size of a test improvement. It does not justify repeated test-driven search. The harder test-bank construction and finite sample variation also prevent interpreting overall validation accuracy as a test-score estimate. The test result is descriptive evidence on this course bank; generalization to an independent audit bank remains to be checked.

6. Resources and compliance

The complete uncompressed inference folder contains a 306806-byte checkpoint and a 377-byte configuration: **307183 bytes (~0.293 MiB)**, below 16 MiB. Inference uses NumPy plus the unchanged feature extraction/parser code. It does not consult references, use IDs as features, retrain, call a model/API or generate new solutions. Reordered/subset inputs, duplicate questions, new IDs and empty banks passed contract checks.

On Windows with an Intel Core Ultra 9 275HX, Python 3.12 and four threads, a fresh complete test prediction process took 0.610 s and a separate scoring process took 0.304 s, **0.914 s total**. This includes startup, imports, input/asset loading and preprocessing. Local results are comfortably below 30 s, but reference-Xeon eligibility is ultimately determined on the reference machine. The contextual search took ~327 s locally; the additional objective/RBF/text/bootstrap experiments took ~24/37/48/130 s respectively. Training used CPU only.

Official JSONs, asset hashes and wall times are saved under `code/runs/final_evaluation/`. The checkpoint SHA-256 is `cc8483cb50c6b7287613897b5b78178b2accd0780888b5ac227d46f1fc984537`. Original frozen files and their earlier results remain untouched.

7. Reproduction

Use Python 3.12 and install `code/requirements-new-experiments.txt`. On Windows, clone with `git -c core.autocrlf=false clone ...` so the official data hashes retain the original line endings. Run from `code/`, with `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `MKL_NUM_THREADS` set to 4:

```
python predict.py --method submission_candidate.py --checkpoint runs/final_candidate --split
test --output runs/reproduced_test_predictions.json
python score_predictions.py --split test --predictions runs/reproduced_test_predictions.json
--output runs/reproduced_test_score.json
```

Expected official accuracy: `83.94004282655246`. Retraining is unnecessary for evaluation. For the already selected train-only configuration:

```
python train_candidate.py --output runs/retrained_context_candidate.npz
```

This command refuses to overwrite an existing output. The development search can be reproduced with `experiment_bilinear.py`; further commands and experiment controls are in `code/NEW_EXPERIMENTS.md` and `code/OPTIMIZATION_ROUND2.md`. Do not use the test bank for any new parameter selection.

We reran this fixed train-only command into a separate directory and confirmed that every checkpoint array was bitwise identical to the frozen asset. The record is in `code/runs/reproduction_check/verification.json`; this check did not change the model or trigger another test run.

A later isolated submission audit reproduced all 467 frozen choices with reference files absent during inference, network access blocked and training imports prohibited. Only the separate scorer received references; it again returned 392/467. No parameter or method selection used this reproduction. Its record is in `code/runs/submission_audit/reproduction_audit.json`.

8. AI assistance and reuse

The original project used Claude coding assistance, as recorded in its preserved report. OpenAI Codex proposed and implemented the contextual extension and subsequent experiments, ran train/validation development and the authorized frozen test evaluation, authored verification scripts and drafted this report. The student must review and understand the implementation and is responsible for the submitted work and claims. Supplied GSM8K/Qwen data and licenses are retained. No external dataset or additional pretrained weights were introduced.